PROGRAMMING FUNDAMENTALS – FINAL EXAM PRACTICE SHEET

Made by: Mahad (BSSE – 2025)

Linkedin : www.linkedin.com/in/codewithmahad
Github : https://github.com/mahad2006

Built to practice logic, arrays, sorting, and patterns – All in one place!

LEVEL 0 – BASIC I/O & MATH FOUNDATION (Beginner logic building, no arrays yet)

---

1. Take the user's name and age as input, and display a greeting message.
   ➤ Ask the user for their name and age. Then display a message like: "Hello [name], you are [age] years old."

2. Take three numbers as input and calculate their sum.
   ➤ Input three integers from the user and print their total.

3. Swap the values of two numbers using a third variable.
   ➤ Store one value temporarily to help switch their positions.

4. Swap the values of two numbers without using a third variable.
   ➤ Use arithmetic operations like addition/subtraction or XOR for swapping.

5. Build a basic calculator using if-else or switch statements.
   ➤ Accept two numbers and an operator (+, -, *, /), then display the result based on the selected operation.

6. Check whether a given number is even or odd.
   ➤ Use modulo (%) operator. If number % 2 == 0, it's even; otherwise odd.

7. Find the largest number among three given numbers.
   ➤ Use if-else or nested conditions to compare the three numbers.

8. Check whether a given year is a leap year.
   ➤ A leap year is divisible by 4, not by 100 unless it's also divisible by 400.

9. Count how many digits are present in a given number.
   ➤ Divide the number by 10 repeatedly and count how many times until it becomes 0.

10. Calculate the sum of digits in a given number.
    ➤ Extract each digit using %10, add it to sum, and remove the digit using /10.

LEVEL 1 – LOOP & NUMBER-BASED LOGIC (Core loop logic mastery)

---

11. Print numbers from 1 to N using all three loop types (for, while, do-while).
    ➤ Use each loop to print numbers starting from 1 up to N.

12. Print numbers from N to 1.
    ➤ Use a loop to display numbers in reverse order.

13. Calculate the sum of the first N natural numbers.
    ➤ Add all numbers from 1 to N using a loop or formula: n(n+1)/2.

14. Calculate the factorial of a number (1 to N).
    ➤ Multiply all numbers from 1 to N.

15. Reverse the digits of a given number.
    ➤ Build a new number by taking digits from the end.

16. Check whether a number is a palindrome.
    ➤ A number that reads the same backward. Example: 121, 1331.

17. Check whether a number is an Armstrong number.
    ➤ A number equal to the sum of its digits each raised to the power of the number of digits.

18. Generate the first N terms of the Fibonacci sequence.
    ➤ Fibonacci starts with 0, 1. Each term = sum of the previous two.

19. Check whether a number is prime.
    ➤ A number with only two factors: 1 and itself.

20. Print all prime numbers from 1 to N.
    ➤ Check each number between 1 to N if it’s prime and print it.

21. Calculate the GCD (HCF) of two given numbers.
    ➤ Greatest Common Divisor is the highest number that divides both. Use loop or Euclidean algorithm.

22. Check whether a number is a perfect number.
    ➤ A number whose proper divisors (excluding itself) add up to the number.

23. Calculate power of a number without using pow().
    ➤ Multiply the base by itself exponent number of times.

24. Convert a decimal number to binary manually.
    ➤ Keep dividing by 2 and store remainders. Reverse them to get binary.

25. Convert a binary number to decimal manually.
    ➤ Multiply each bit with powers of 2 and add them up.

LEVEL 2 – PATTERN PRINTING CHALLENGES

---

26. Print a square pattern of asterisks.
    ➤ Example for N=3: *** \n *** \n ***

27. Print a right-angled triangle pattern using asterisks.
    ➤ First row has 1 star, second has 2, and so on.

28. Print a number triangle pattern (1, 22, 333...).
    ➤ Each row prints its number repeatedly.

29. Print Floyd's triangle pattern.
    ➤ Increasing numbers in a triangle shape: 1 \n 2 3 \n 4 5 6...

30. Print a pyramid-shaped pattern using asterisks.
    ➤ Centered triangle with increasing stars.

31. Print Pascal's triangle.
    ➤ A triangle of binomial coefficients.

32. Print an alphabet triangle pattern (A, AB, ABC...).
    ➤ Each row prints increasing sequence of characters starting from A.

33. Print a hollow square pattern.
    ➤ Stars at borders only, empty inside.

34. Print a checkerboard pattern using 1s and 0s.
    ➤ Alternate 1s and 0s row-wise or cell-wise.

35. Print a triangle pattern using prime numbers only.
    ➤ Print primes in triangle format row by row.

LEVEL 3 – 1D ARRAY BASICS

---

36. Take N numbers as input and store them in an array.
    ➤ Ask user for N, then input each number and store it in an array.

37. Print all the elements of the array.
    ➤ Loop through and display each element one by one.

38. Calculate the sum of all array elements.
    ➤ Add each element using a loop and print the result.

39. Find the maximum and minimum values in the array.
    ➤ Compare each element to track max and min values.

40. Count how many elements are even and how many are odd.
   ➤ Use modulus operator to check and count evens/odds.

41. Search for a specific element in the array.
   ➤ Use a linear search to find if the element exists and its index.

42. Reverse the elements of the array.
   ➤ Swap first with last, second with second-last, and so on.

43. Copy one array into another.
   ➤ Create a new array and copy each element from the original array.

44. Find the second largest element in the array.
   ➤ Track two largest values while looping through the array.

45. Remove duplicate elements from the array.
   ➤ Use nested loops or a hash map to skip repeating values.

46. Count the frequency of each element.
   ➤ Count how many times each element occurs in the array.

47. Print only the unique elements from the array.
   ➤ Display elements that appear exactly once.

48. Find the element that appears once while all others appear twice.
   ➤ Use XOR trick or count each element's frequency.

49. Count how many elements are greater than the average.
   ➤ First find average of all elements, then count how many are greater than it.

LEVEL 4 – ARRAY ROTATION & COMBINATIONS

---

50. Left-rotate the elements of an array.
   ➤ Move each element one step to the left and send the first element to the end.

51. Right-rotate the elements of an array.
   ➤ Move each element one step to the right and bring the last element to the front.

52. Merge two arrays into a third array.
   ➤ Combine both arrays one after the other into a new array.

53. Find common elements between two arrays.
   ➤ Print elements that exist in both arrays.

54. Check whether an array is a palindrome.
   ➤ Same forward and backward (e.g., [1, 2, 3, 2, 1]).

55. Move all zero elements to the end.

➤ Keep the order of other elements the same, just move 0s to the right.

56. Replace each array element with its square.
➤ Multiply each element by itself.

57. Rearrange array in max-min form.
➤ Alternate the largest and smallest elements in the array.

58. Count how many elements in the array are prime.
➤ Use prime-check logic for each element in the array.

59. Calculate the sum of digits of all array elements.
➤ Add the digits of each number in the array, then print the total.

60. Count how many array elements are palindromes.
➤ Reverse each number and compare with original to check.

LEVEL 5 – SORTING & SEARCHING

---

61. Sort the array using Bubble Sort.
➤ Repeatedly swap adjacent elements if they're in the wrong order.

62. Sort the array using Selection Sort.
➤ Repeatedly pick the smallest element and put it at the beginning.

63. Sort the array using Insertion Sort.
➤ Insert each element into its correct place within the sorted part of the array.

64. Sort the array in descending order.
➤ Modify any sort algorithm to sort from largest to smallest.

65. Sort even numbers first, then odd numbers.
➤ Separate evens and odds, then sort each and combine.

66. Sort digits of an array and form the largest possible number.
➤ Combine digits, sort in descending order, and print the final number.

67. Binary Search in a sorted array.
➤ Divide the array in half each time to find the target element.

68. Merge two sorted arrays into one sorted array.
➤ Use two pointers and compare elements to merge correctly.

69. Remove duplicate elements from a sorted array.
➤ Skip over repeating elements while traversing the array.

LEVEL 6 – FINAL COMBO CHALLENGES (EXAM-STYLE)

---

70. Check whether a number is pandigital (contains all digits from 0 to 9).
    ➤ Convert number to string and check if it contains every digit at least once.

71. Rotate the digits of a number to the left.
    ➤ Shift all digits left by one place and move the first digit to the end.

72. Rotate the digits of a number to the right.
    ➤ Shift all digits right by one place and bring the last digit to the start.

73. Find and print all Armstrong numbers from 1 to N.
    ➤ Check each number: sum of each digit raised to the power of number of digits should equal the number.

74. Calculate and print the sum of the series: $x/1! + x^2/2! + x^3/3! + ... + x^n/n!$
    ➤ Implement logic using loops and factorial calculations.

75. Count how many numbers from 1 to N are divisible by both 3 and 5.
    ➤ Check each number: if divisible by 15, increase the count.

76. Print the factorial of each element in the array.
    ➤ Loop through the array and calculate factorial for each number.

77. Find the first non-repeating element in the array.
    ➤ Count frequency of all elements, then find the first with count 1.

78. Count how many digits are used in the entire array.
    ➤ Break all numbers into digits and count total occurrences.

79. Count how many elements in the array are perfect numbers.
    ➤ Use perfect number check logic for each element.

80. Check whether the array is sorted in ascending order or not.
    ➤ Compare each element to the next; if any is greater than the next, it's not sorted.

81. Find all leaders in the array (element > all elements to its right).
    ➤ Traverse from right to left; if current element is greater than max seen so far, it’s a leader.

LEVEL 7 – ADVANCED LOGIC & BRAIN TWISTERS (Top 0.001% Coder Zone – with explanations)

---

82. Check if a number is a Strong Number
    ➤ A number is called a Strong Number if the sum of the factorials of its digits equals the number itself.
    Example: 145 → 1! + 4! + 5! = 145

83. Print all Happy Numbers between 1 and N

➤ A Happy Number is a number that eventually reaches 1 when replaced by the sum of the squares of its digits repeatedly.
If it loops endlessly in a cycle not including 1, it's called an Unhappy Number.
Example: 19 → $1^2 + 9^2 = 82$ → $8^2 + 2^2 = 68$ → ... → 1 ✅

84. Find the longest sequence of consecutive 1s in the binary representation of a number
➤ Example: 222 → binary: 11011110 → longest streak of 1s = 4

85. Count the number of 1s in the binary form of a number (Set Bits)
➤ Example: 13 → binary: 1101 → Set bits = 3

86. Determine if two numbers are Co-Prime
➤ Two numbers are co-prime if their GCD is 1.
Example: (8, 15) → GCD = 1 → co-prime ✅

87. Find the smallest positive number divisible by all numbers from 1 to N
➤ This is the Least Common Multiple (LCM) of numbers from 1 to N.
Example: N = 5 → LCM of 1,2,3,4,5 = 60

88. Find the first N prime numbers that are also palindromes
➤ A number is a palindrome if it reads the same forward and backward.
Example: 2, 3, 5, 7, 11, 101, 131...

89. Given a number, find the next greater number using the same digits
➤ Example: 1234 → Next: 1243 | If not possible, return -1

90. Print all subsets (Power Set) of a given array
➤ For array [1,2], subsets: [], [1], [2], [1,2]
Total = 2^n subsets

91. Count total subarrays with an even sum
➤ Given an array, count how many contiguous subarrays have a sum that is even

92. Find the Longest Increasing Subsequence (LIS) in an array
➤ A subsequence is not necessarily contiguous.
Example: [10, 22, 9, 33] → LIS: [10, 22, 33] → Length = 3

93. Find the Equilibrium Index in an array
➤ An index where sum of elements on the left = sum on the right
Example: [1, 3, 5, 2, 2] → index 2 → 1+3 = 2+2

94. Check if a number can be expressed as a sum of two squares
➤ Example: $5 = 1^2 + 2^2$ → ✅
Try all $(a^2 + b^2)$ pairs where a, b are ≤ √n

95. Implement the Sieve of Eratosthenes to generate all primes ≤ N
➤ Efficient algorithm for finding all primes up to a limit N using a boolean array

96. Find all triplets in the array that sum up to a given target
➤ Given arr[] and sum = target, find all combinations of 3 numbers such that arr[i] + arr[j] + arr[k] = target

97. Rotate a square matrix (2D array) 90° clockwise

➤ You'll need to transpose the matrix and then reverse each row

98. Count the number of pairs in an array with a given difference
➤ Given arr[] and difference = k, find pairs (a, b) such that |a - b| = k

99. Check if array contains duplicates within distance k
➤ Return true if any two identical elements exist at most k indices apart

100. Find the majority element in an array
➤ The element that appears more than n/2 times
Use counting logic or Boyer-Moore Voting Algorithm

101. Check if a number can be written as the sum of consecutive natural numbers
➤ Example: 15 → 1+2+3+4+5 = 15 ✅ | Try all sequences until the sum exceeds the number

---

LEVEL 8 – NIGHTMARE MODE (🔥Only for the Brave – Based on Loops + Arrays + Math)

---

102. Find the length of the longest increasing sequence of numbers in the array.
➤ Use a loop to track how long each increasing streak continues.

103. Find the subarray with the highest sum using only loops.
➤ Try every possible subarray and track the maximum sum found.

104. Count how many subarrays have an even number of elements and an even sum.
➤ Generate all subarrays, check length and sum conditions.

105. For a given N, print all combinations of three numbers that sum to N.
➤ Use three nested loops: if i + j + k == N, print the combination.

106. Create a frequency table of digit sums of all array elements.
➤ For each element, find sum of digits and count how often each digit sum occurs.

107. Given an array, count how many elements are smaller than the element to their right.
➤ For each index i, compare arr[i] < arr[i+1].

108. Reverse an array using loop without using any built-in functions.
➤ Swap first and last, second and second-last manually using a loop.

109. Find the element for which the absolute difference with the average is minimum.
➤ Find average first, then loop to find the closest.

110. Count how many numbers in the array are divisible by the sum of their digits.
➤ For each element, calculate digit sum and check divisibility.

111. Find the number of subarrays where all elements are even.
➤ Try all subarrays and check if all elements are divisible by 2.

TIPS TO SCORE FULL MARKS – FINAL EXAM

Just a few things that are really helping me while practicing PF, thought I'd share:

- Try to practice daily, even if it's just 3–5 questions at night — it really adds up over time.
- Don't just focus on the syntax. Try to understand *why* each step works — that's where the real learning happens.
- Dry run your code on paper, especially for loops and tricky index stuff — it makes things way clearer.
- Keep your code neat. Use proper variable names so it's easier to read and debug later.
- Don't skip the tough questions. They might take time, but they build confidence super fast.
- Try to guess the output before running your code — great way to improve debugging skills.
- And yeah, push your work to GitHub if you can. It's a good way to track your progress and revisit stuff later.

END OF SHEET – MASTER THIS = YOU'RE UNSTOPPABLE
